



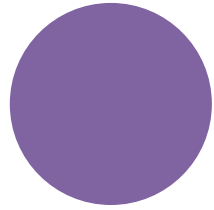
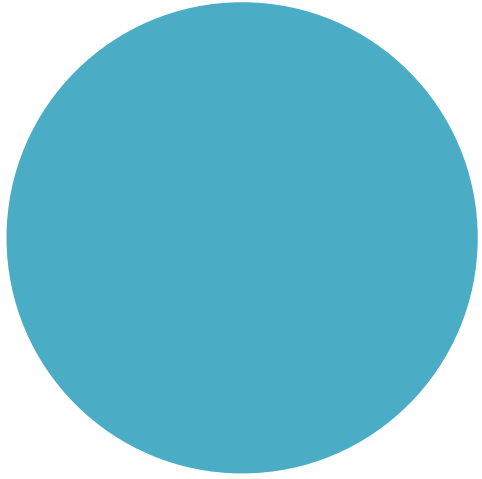
Mobile Devices and Applications

Unit 7 – Android Asynchronous Operations



Summary

- Threads
- Handlers
- Callback / Callable
- Executor / Runnable
- Network access
 - Socket
 - HTTP
 - Web services



ASYNC OPERATIONS



A good developer knows he needs to off load heavy tasks to a worker Thread to avoid clogging the Main Thread and allow a smoother user experience and avoid ANR.

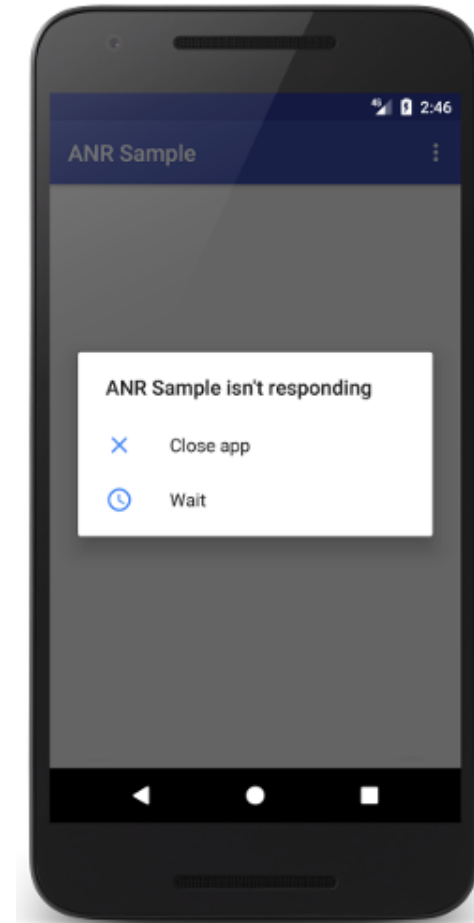
FOUNDATIONAL CONCEPT

Asynchronous operations

- Used for heavily resource consuming activities
- They must be executed on an independent thread
 - the main thread or UI thread must not be blocked
- Two main problems:
 - thread communications, how to send input and output data
 - publishing progress: how to interact with the controls associated with the main thread
- **NetworkOnMainThreadException**

Android ANR

- Application **N**ot **R**esponding
- Trigger conditions:
 - while your activity is in the foreground, your app has not responded to an input event or BroadcastReceiver (such as key press or screen touch events) within 5 seconds.
 - while you do not have an activity in the foreground, your BroadcastReceiver hasn't finished executing within a considerable amount of time.



Diagnosing ANRs

The app is doing slow operations involving I/O on the main thread.

The app is doing a long calculation on the main thread.

The main thread is doing a synchronous binder call to another process, and that other process is taking a long time to return.

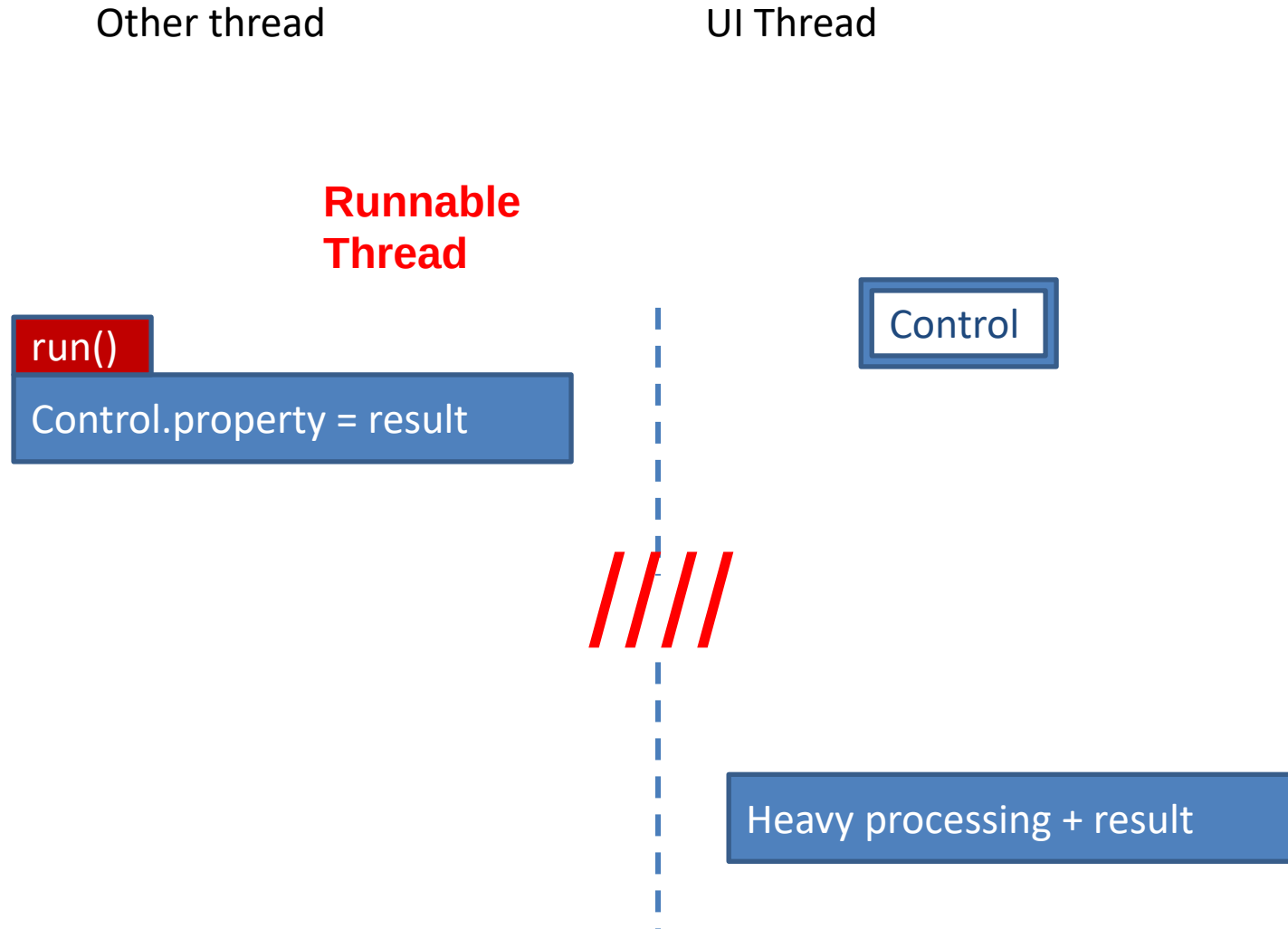
The main thread is blocked waiting for a synchronized block for a long operation that is happening on another thread.

The main thread is in a deadlock with another thread, either in your process or via a binder call

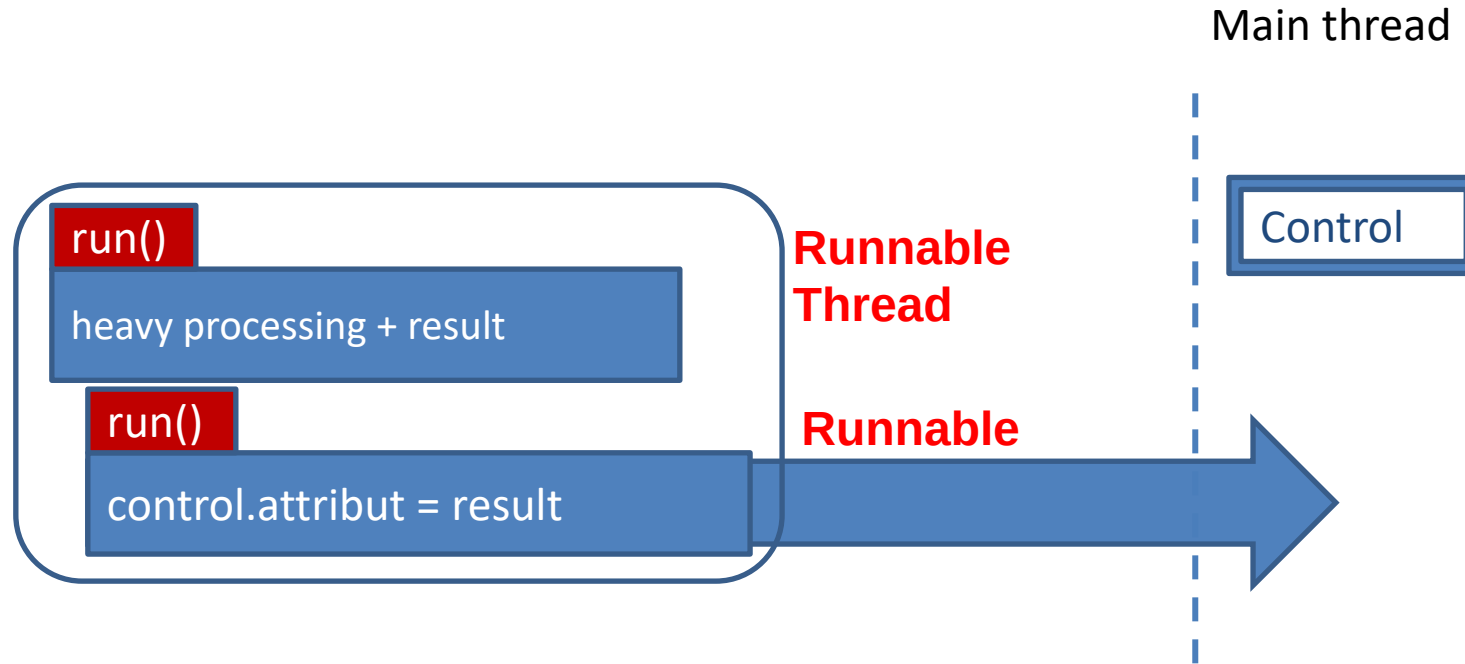
Asynchronous operations

- Java
 - **Thread** class
 - **Runnable** interface
- Android
 - Methods
 - **synchronized(this)**
 - **runOnUiThread(Runnable)** (**Activity** class) – switches between threads
 - **post(Runnable)** (**View** class) – a new thread managed by a handler
 - **postDelayed(Runnable, long)** (**View** class)
 - **Handler** class
 - **AsyncTask** class

Asynchronous operations



Asynchronous operations



Activity#runOnUiThread(**Runnable**)
View#post(**Runnable**)
View#postDelayed(**Runnable**, long)

Handler & Looper

- Communication with main thread
- Manages two types of messages:
 - what – already created messages
 - Runnable – messages that contain code to be executed
- Handles messages processed by the Looper:
 - extend Handler class with our own custom implementation and override handleMessage() method
 - create Handler instance with constructor providing custom Handler.Callback (which has handleMessage method)

Handler

`post()` – causes the Runnable object to be added to the message queue

`dispatchMessage()` – handle system message on the thread where it is called

`sendMessage()` – pushes a message onto the end of the message queue of the thread where the Handler was created;

`obtainMessage()` – returns a new Message from the global message pool;

`handleMessage()` – used to handle the message into subclasses

`getMainLooper()` – returns the Looper for the main thread

Handler workflow

Client initializes Handler and Looper

Client sends messages to Handler

Handler posts messages to Looper's MessageQueue

Looper processes messages in MessageQueue

When message is ready to be processed, Looper sends it back to Handler, which can handle message.

Update UI

```
Handler(Looper.getMainLooper()).post(() -> updateUI());
```

ExecutorService vs. ThreadPoolExecutor

- Used for short background task
- Managing thread pools efficiently (fixed, cached, scheduled), reusing threads.

Thread safety and synchronization

- Avoid race conditions in shared mutable state:
 - synchronized
 - Volatile
 - AtomicTypes
 - ReentrantLock

AsyncTask class -obsolete

- Abstract class AsyncTask<paramsType, progressType, resultType>
- Implemented methods:
 - doInBackground(paramsType... pars)
 - onProgressUpdate(progressType...pars)
 - publishProgress()
 - onPostExecute(resultType par)
 - onPreExecute()
- Call:
 - execute(params) method
- Does not accept primitives, only object type values

AsyncTask class

- `onPreExecute()`
 - The first method executed after initializing a call
- **`doInBackground()`**
 - Heavily resource processing
 - Runs asynchronously
- `onProgressUpdate()`
 - It is constantly executed for displaying the progress of the `doInBackground` operations
 - **`publishProgress()`**
- `onPostExecute()`
 - Called after the `doInBackground` has finished processing
 - Receives the result returned by the processing stage

Calls for the AsyncTask class

UI THREAD

- 1. `onPreExecute()`
- 4. `onProgressUpdate()`
- 5. `onPostExecute()`

SECONDARY THREAD

- 2. `doInBackground()`
 - 3. `publishProgress()`



AsyncTask class

```
class HeavyTask extends AsyncTask<String, Double, Bitmap>
{
    @Override
    protected void onPreExecute() {
        super.onPreExecute();
    }

    @Override
    protected Bitmap doInBackground(String... params) {
        publishProgress((Double)values);
        return null;
    }

    @Override
    protected void onProgressUpdate(Double... values) {
        super.onProgressUpdate(values);
    }

    @Override
    protected void onPostExecute(Bitmap bitmap) {
        super.onPostExecute(bitmap);
    }
}
```

AsyncTask class

onPreExecute() method:

```
@Override
protected void onPreExecute() {
    super.onPreExecute();
    //initializing objects
}
```

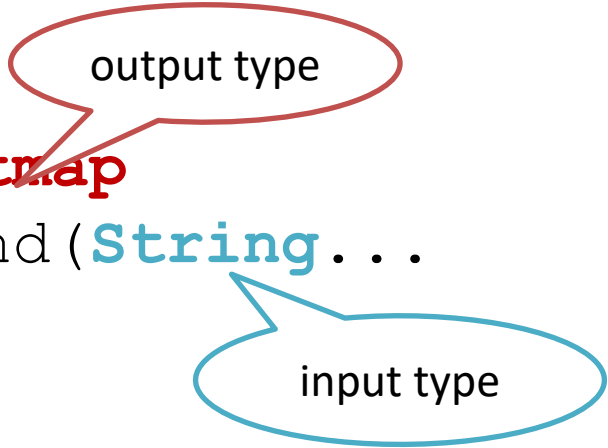
Important aspects:

- initialization of anything needed to display the progress
- called once before the doInBackground() method

AsyncTask class

doInBackground() method:

```
@Override
protected Bitmap
doInBackground(String...
params)
{
    //heavy processing;
    publishProgress(value);
}
```



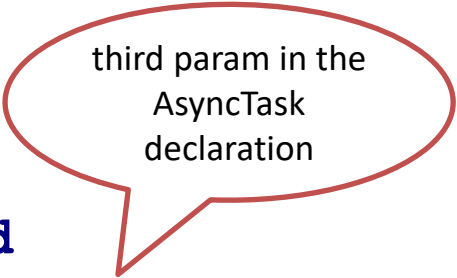
Important aspects:

- variable number of arguments, an array of strings
- parameters are sent from the UI thread
- the execute method of the AsyncTask will receive a variable number of params
- the method always returns a parameter of the same type declared by the AsyncTask

AsyncTask class

onPostExecute() method:

```
@Override
protected void
onPostExecute(Bitmap bitmap) {
    super.onPostExecute(bitmap);
    //processing the result
}
```



third param in the
AsyncTask
declaration

Important aspects:

- The parameter in the list is the one returned by the doInBackground method
- This method runs on the UI thread so it can interact with the UI controls

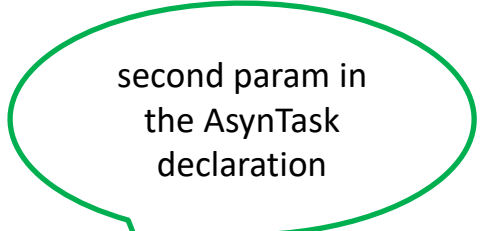
AsyncTask class

onProgressUpdate() method:

```
@Override
protected void
onProgressUpdate(Double... values)
{

    super.onProgressUpdate(values);
    //displaying the progress

}
```



second param in
the AsyncTask
declaration

Important aspects:

- The values are sent by calling the `publishProgress()` method from the `doInBackground()`
- This method runs on the UI thread so it can interact with the UI controls

Downloading files

- System Service **DownloadManager**
 - `getSystemService(DOWNLOAD_SERVICE)`
- Initializing a new request object
- Starting the request
 - *DownloadManager#enqueue()*
- Checking the process state
 - *DownloadManager#query()*
- Finishing the process
 - BroadcastReceiver for signalling events
 - Cursor for reading the data

Resources

- C. Boja, C. Ciurea, M. Doinea – Android Mobile Applications: A Practical Development Guide, ASE Press, 2015, 404 pp., ISBN 978-606-34-0033-9
- S. Komatineni, D. MacLean – Pro Android 4, Apress, 2012
- R. Meier – Professional Android 4 Application Development, Wiley, 2012
- P. Pocatilu, Programarea dispozitivelor mobile, Editura ASE, 2012
- P. Pocatilu, I. Ivan ș.a. – Programarea aplicațiilor Android, Editura, ASE, 2015
- <http://developer.android.com>